

Correction des Questions Théoriques – TP 3

Guidage par Vision Artificielle et Intégration Système sous ROS

2

Filière Robotique

Mai 2026

Table des matières

1	Exercice 1 : Traitement d’Images et Perception (Vision)	2
1.1	Question 1 : Intérêt et avantages de l’espace colorimétrique HSV	2
1.2	Question 2 : Analyse mathématique du filtre de persistance temporelle	2
1.3	Question 3 : Analyse Multilevel et Filtrage Géométrique	2
1.4	Question 4 : Estimation de l’erreur en mode dégradé (Suivi Mono-ligne)	3
2	Exercice 2 : Contrôle par Correcteur PD et Gestion Événementielle	4
2.1	Question 1 : Signe négatif de la commande angulaire et apport du terme Dérivé .	4
2.2	Question 2 : Analyse du profil de vitesse linéaire dynamique en mode NOMINAL	4
2.3	Question 3 : Exploitation de la mémoire géométrique en état de DETRESSE . . .	4
3	Exercice 3 : Intégration Système et Communication Inter-Nœuds	5
3.1	Question 1 : Cartographie des flux de communication ROS 2	5
3.2	Question 2 : Analyse du protocole de service “Secours Solaire”	5

1 Exercice 1 : Traitement d'Images et Perception (Vision)

1.1 Question 1 : Intérêt et avantages de l'espace colorimétrique HSV

La méthode `image_callback()` convertit le flux du format natif BGR vers l'espace **HSV** (*Hue, Saturation, Value*) avant d'appliquer le seuillage par plage `cv2.inRange()`.

Raison technique : L'espace BGR mélange intimement l'information colorimétrique pure et l'intensité lumineuse sur ses trois canaux (R, G, B). L'espace HSV, quant à lui, **découple totalement la teinte chromatique (Hue)** de la pureté de la couleur (Saturation) et de la brillance globale (Value).

Avantage concret en robotique réelle : En conditions réelles, le robot est confronté à des variations d'éclairage continues (zones d'ombres, passages nuageux, reflets de projecteurs). En HSV, on peut isoler la couleur jaune des lignes en verrouillant une plage très étroite sur le canal **H** (la teinte intrinsèque), tout en ouvrant très largement les tolérances sur le canal **V** (la luminosité). Le système devient ainsi hautement robuste et insensible aux fluctuations d'éclairage ambiant, ce qui serait impossible à réaliser de manière stable en BGR.

1.2 Question 2 : Analyse mathématique du filtre de persistance temporelle

Calcul opéré par `np.sum(... == 255, axis=0)` : Le code inspecte l'historique des N derniers masques binaires stockés dans la file d'attente circulaire (**deque**). Pour chaque coordonnée de pixel (x, y) de la matrice image, l'opération compte de manière temporelle (le long de l'axe `axis=0`) le nombre de fois où ce pixel précis a été évalué à la valeur blanche 255 (c'est-à-dire détecté comme faisant partie de la piste).

Rôle et immunité anti-clignotement : Ce mécanisme agit comme un **filtre de persistance ou de vote majoritaire temporel**. Si la ligne jaune disparaît subitement pendant une seule trame d'image à cause d'un reflet spéculaire intense ou d'un bruit électronique, les $N - 1$ trames mémorisées dans la file maintiennent une valeur de somme élevée pour ces pixels. Le seuillage ultérieur préserve la structure de la ligne, immunisant complètement le robot contre les micro-pertes furtives de signal (clignotement) et évitant les déclenchements intempestifs du mode de détresse.

1.3 Question 3 : Analyse Multilevel et Filtrage Géométrique

Diminution de la largeur mémorisée ($L1 \rightarrow L2 \rightarrow L3$) : Cette décroissance est la conséquence directe de l'**effet de perspective optique** (projection 3D du monde réel vers le plan image 2D de la caméra). Les lignes parallèles de la piste convergent visuellement vers un point de fuite situé vers le haut de l'image. Ainsi, à mesure que l'on s'élève vers les horizons lointains ($Y_{L3} = 170$), la distance en pixels séparant les deux lignes se réduit naturellement.

Fonctionnement de `filtrer_paire_par_largeur` : Cette méthode mesure l'écartement horizontal en pixels entre deux contours jaunes segmentés sur un même horizon. Elle compare cette valeur à la largeur cible mémorisée théorique. Si un reflet jaune parasite apparaît au sol sur le côté de la piste, l'écartement mesuré entre ce reflet et la vraie ligne sera mathématiquement aberrant par rapport au modèle géométrique attendu, permettant au script d'éliminer immédiatement la fausse paire.

Intérêt du filtre passe-bas sur la largeur cible : La mise à jour ($0.9 * \text{largeur_cible} + 0.1 * \text{largeur_calculée}$) applique un lissage numérique. Il évite que la largeur de référence ne subisse des sauts brutaux à cause de bruits de détection sur un pixel isolé, garantissant une convergence douce et stable vers la géométrie réelle de la route.

1.4 Question 4 : Estimation de l'erreur en mode dégradé (Suivi Mono-ligne)

Lorsqu'un virage est extrêmement serré, l'une des lignes (par exemple la ligne gauche) sort totalement du champ de vision de la caméra (`l1_ok = False`). Le script exploite alors la **propriété géométrique de symétrie du couloir** : il s'appuie sur la position de la seule ligne restante (la ligne droite) et applique un décalage horizontal fixe vers la gauche égal à la **demi-largeur théorique de la piste** apprise et mémorisée pour cet horizon précis :

$$C_{\text{estimé}} = C_{\text{droit}} - \frac{\text{self.largeurs_memoire}[L1]}{2} \quad (1)$$

L'erreur de centrage par rapport à l'axe optique de la caméra est ensuite normalement calculée à partir de ce centre reconstruit.

2 Exercice 2 : Contrôle par Correcteur PD et Gestion Événementielle

2.1 Question 1 : Signe négatif de la commande angulaire et apport du terme Dérivé

Signification du signe négatif ($\text{angular_z_pid} = -(P + D)$) : Ce signe correspond au formalisme mathématique de la **rétroaction négative (boucle fermée)**. Selon la convention de repère standard sous ROS 2 (axe Z pointant vers le haut), une vitesse angulaire ω_z positive engendre une rotation vers la gauche (sens trigonométrique). Si l'erreur de centrage est positive, cela signifie que la piste est décalée vers la droite du robot. Le robot doit donc tourner à droite (ce qui implique une vitesse ω_z négative) pour corriger sa trajectoire. Le signe moins permet de ramener l'erreur vers zéro.

Information apportée par le terme Dérivé (D) : Lorsque le robot sort d'une courbe à grande vitesse et se réaligne avec l'entrée d'une ligne droite, l'erreur de centrage diminue à une vitesse extrêmement élevée vers zéro. La dérivée $\frac{de}{dt}$ prend alors une valeur massive opposée au sens de rotation actuel. Le terme dérivé agit comme un **amortisseur prédictif** : il anticipe le centrage parfait et applique un contre-couple moteur pour freiner la rotation avant que l'erreur ne s'annule, évitant ainsi au robot d'osciller au-delà de l'axe de la ligne droite.

2.2 Question 2 : Analyse du profil de vitesse linéaire dynamique en mode NOMINAL

En mode **NOMINAL**, la vitesse linéaire (`msg.linear.x`) est calculée pour être inversement proportionnelle à l'amplitude de l'erreur de centrage courante.

Fonctionnement : Lorsque le robot est parfaitement centré (erreur proche de zéro), la vitesse linéaire est maximale pour optimiser le temps de parcours. Dès que l'erreur augmente (ce qui annonce un désalignement ou l'approche d'une courbe prononcée), la loi de commande réduit automatiquement la vitesse linéaire.

Intérêt pour la préservation mécanique : Cette approche ménage les actionneurs et le châssis physique : elle évite d'engager le robot à pleine vitesse dans des courbes serrées, ce qui élimine les risques de patinage cinématique des roues, limite les forces centrifuges sur la structure et réduit l'usure prématurée des moteurs due à des corrections d'attitude trop brutales.

2.3 Question 3 : Exploitation de la mémoire géométrique en état de DETRESSE

Le robot enregistre en continu deux variables clés en fonctionnement normal : `self.dernier_sens_virage` (indiquant si la dernière courbe s'orientait à gauche ou à droite) et `self.angle_accumule`.

Comportement en état de DETRESSE : Si le marquage au sol disparaît brusquement (perte totale de la piste), le robot bascule en état de **DETRESSE**. Au lieu de s'arrêter ou de chercher au hasard, il exploite cette mémoire court-terme pour **ordonner un pivotement aveugle mais logique** dans la direction enregistrée par `self.dernier_sens_virage`.

Utilité : Ce mécanisme empêche le robot de partir dans la direction opposée (ce qui l'éloignerait définitivement de la piste). Il lui permet d'effectuer un balayage angulaire ciblé dans la zone la plus probable pour recouper la ligne jaune et restaurer le suivi nominal de manière autonome.

3 Exercice 3 : Intégration Système et Communication Inter-Nœuds

3.1 Question 1 : Cartographie des flux de communication ROS 2

Le tableau ci-dessous synthétise l'ensemble des flux d'informations et d'interactions mis en œuvre entre le nœud de traitement d'images (`vision_line_node`) et le nœud de commande cinématique (`control_line_node`) :

TABLE 1 – Matrice d'échange des flux inter-nœuds sous ROS 2

Nom du Flux	Mécanisme	Type de Message	Rôle de la Donnée
<code>/line_geometry</code>	Topic	<i>Message personnalisé</i> ou <code>std_msgs/Float32</code>	Transmet l'erreur de centrage (pixels) calculée sur les différents horizons.
<code>/line_status</code>	Topic	<code>std_msgs/Bool</code> (ou dans le message précédent)	Indique la présence ou la perte des lignes sur les horizons respectifs ($L1, L2, L3_ok$).
<code>/camera/.../set_parameters</code>	Service	<code>rcl_interfaces/SetParameters</code>	Modifie dynamiquement les registres d'exposition face à l'ensoleillement.
<code>/cmd_vel</code>	Topic	<code>geometry_msgs/Twist</code>	Publie les commandes de vitesses cinématiques (v_x, ω_z) finales vers la plateforme robot.

3.2 Question 2 : Analyse du protocole de service “Secours Solaire”

Différence fondamentale entre Topic et Service : L'envoi de commandes sur le topic `/cmd_vel` est un flux d'informations continu, asynchrone, de type *Fire-and-Forget* (le nœud publie sans attendre de confirmation). La modification de l'exposition via le service `set_parameters` est une interaction ponctuelle, bidirectionnelle, basée sur un modèle **Requête-Réponse** avec accusé de réception.

Pourquoi un Service ici ? La reconfiguration matérielle des paramètres d'une caméra (gain, temps d'exposition) est une action de configuration critique et discrète. Le nœud de contrôle doit impérativement s'assurer que le capteur a correctement validé et appliqué les nouveaux paramètres avant de poursuivre son traitement optique, ce qu'un topic ne permet pas de garantir.

Pourquoi un appel asynchrone (`call_async()`) ? Si l'appel de service était bloquant (`call()`), le thread d'exécution du nœud de contrôle se figerait complètement en attendant la réponse réseau de la caméra. Cela interromprait la boucle de sécurité de calcul des vitesses du robot, empêchant toute réaction en cas d'obstacle immédiat. L'appel `call_async()` permet de soumettre la requête en arrière-plan tout en laissant le robot calculer et rafraîchir ses commandes moteurs à haute fréquence.